Index : 230038E

Repo : https://github.com/NileshAm/EN3160-Image-Processing-and-Machine-Vision

Imports

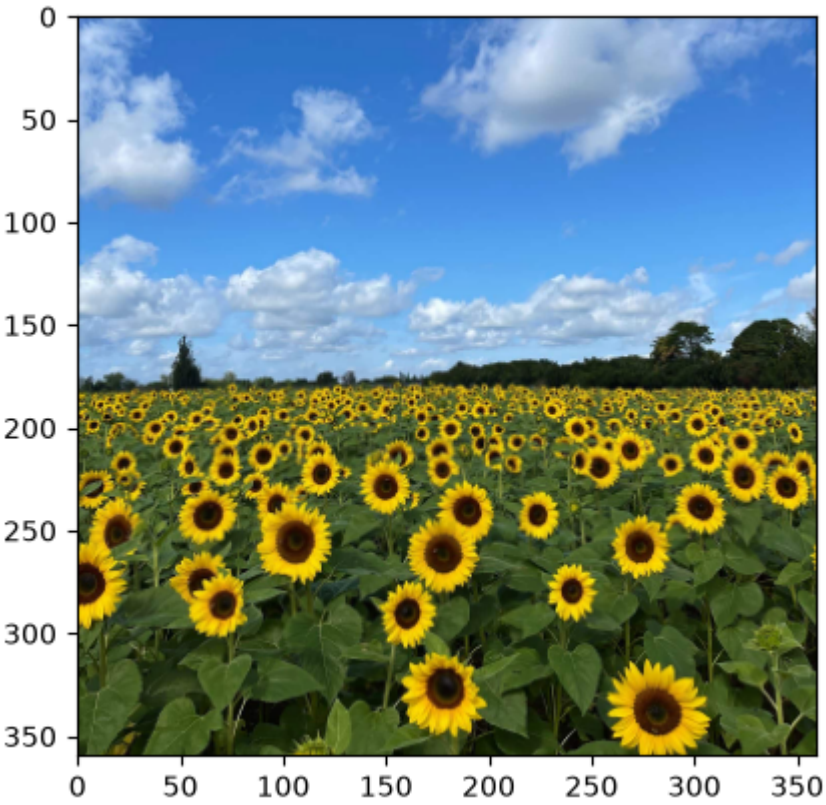
In [1]:
```
import cv2 as cv
import matplotlib.pyplot as plt
from matplotlib.patches import Circle
import numpy as np
from scipy import linalg
from scipy.optimize import minimize
from scipy.ndimage import gaussian_laplace, maximum_filter
```

Question 1

In [2]:
```
img = cv.imread("images/the_berry_farms_sunflower_field.jpeg", cv.IMREAD_
```

In [3]:
```
plt.imshow(cv.cvtColor(img, cv.COLOR_BGR2RGB))
```

Out[3]:
```
<matplotlib.image.AxesImage at 0x7121c3b97bc0>
```



In [4]:
```
grayImg = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
grayImg = grayImg.astype(np.float32) / 255.0
```

In [5]:
```
sigma_min = 1
sigma_max = 12
num_scales = 25
```

```
thes = 0.5
sigmas = np.linspace(sigma_min, sigma_max, num_scales)
```

In [6]:
```
scale_space = np.stack([
    sigma**2 * gaussian_laplace(grayImg, sigma=sigma)
    for sigma in sigmas
], axis=0)

scale_space = np.maximum(scale_space, 0)
```

In [7]:
```
local_max = maximum_filter(scale_space, size=(3, 5, 5))
threshold = thes * scale_space.max()
peaks = (scale_space == local_max) & (scale_space > threshold)

scale_ids, ys, xs = np.where(peaks)
blobs = [(int(x), int(y), float(sigmas[k]), float(np.sqrt(2) * sigmas[k]))
         for k, y, x in zip(scale_ids, ys, xs)]
blobs.sort(key=lambda b: b[3], reverse=True)
```

In [8]:
```
result = img.copy()
for x, y, sigma, radius in blobs:
    cv.circle(result, (x, y), int(round(radius)), (0, 0, 255), 1)

plt.figure(figsize=(10, 10))
plt.imshow(cv.cvtColor(result, cv.COLOR_BGR2RGB))
plt.title('LoG blob detections')
plt.axis('off')
plt.show()

print(f'Sigma range: {sigmas.min():.1f} to {sigmas.max():.1f}')
print(f'Number of scales: {len(sigmas)}')
print(f'Total blobs: {len(blobs)}')
```

LoG blob detections



```
Sigma range: 1.0 to 12.0
Number of scales: 25
Total blobs: 192
```

Question 2

In [9]:

```python
# Generation of a Noisy Point Set Conforming to a Line and a Circle

import numpy as np
from scipy.optimize import minimize
from scipy import linalg
import matplotlib.pyplot as plt

# np.random.seed(0)

N = 100
half_n = N // 2

# Generate noisy circle points
r = 10
x0_gt, y0_gt = 2, 3  # Center
```

```
s = r / 16
t = np.random.uniform(0, 2 * np.pi, half_n)
n = s * np.random.randn(half_n)

x = x0_gt + (r + n) * np.cos(t)
y = y0_gt + (r + n) * np.sin(t)

X_circ = np.hstack((
    x.reshape(half_n, 1),
    y.reshape(half_n, 1)
))

# Generate noisy line points
s = 1.0
m, b = -1, 2

x = np.linspace(-12, 12, half_n)
y = m * x + b + s * np.random.randn(half_n)

X_line = np.hstack((
    x.reshape(half_n, 1),
    y.reshape(half_n, 1)
))

# Combine all points
X = np.vstack((X_circ, X_line))

# Plot points
fig, ax = plt.subplots(1, 1, figsize=(8, 8))

ax.scatter(X_line[:, 0], X_line[:, 1], label='Line')
ax.scatter(X_circ[:, 0], X_circ[:, 1], label='Circle')

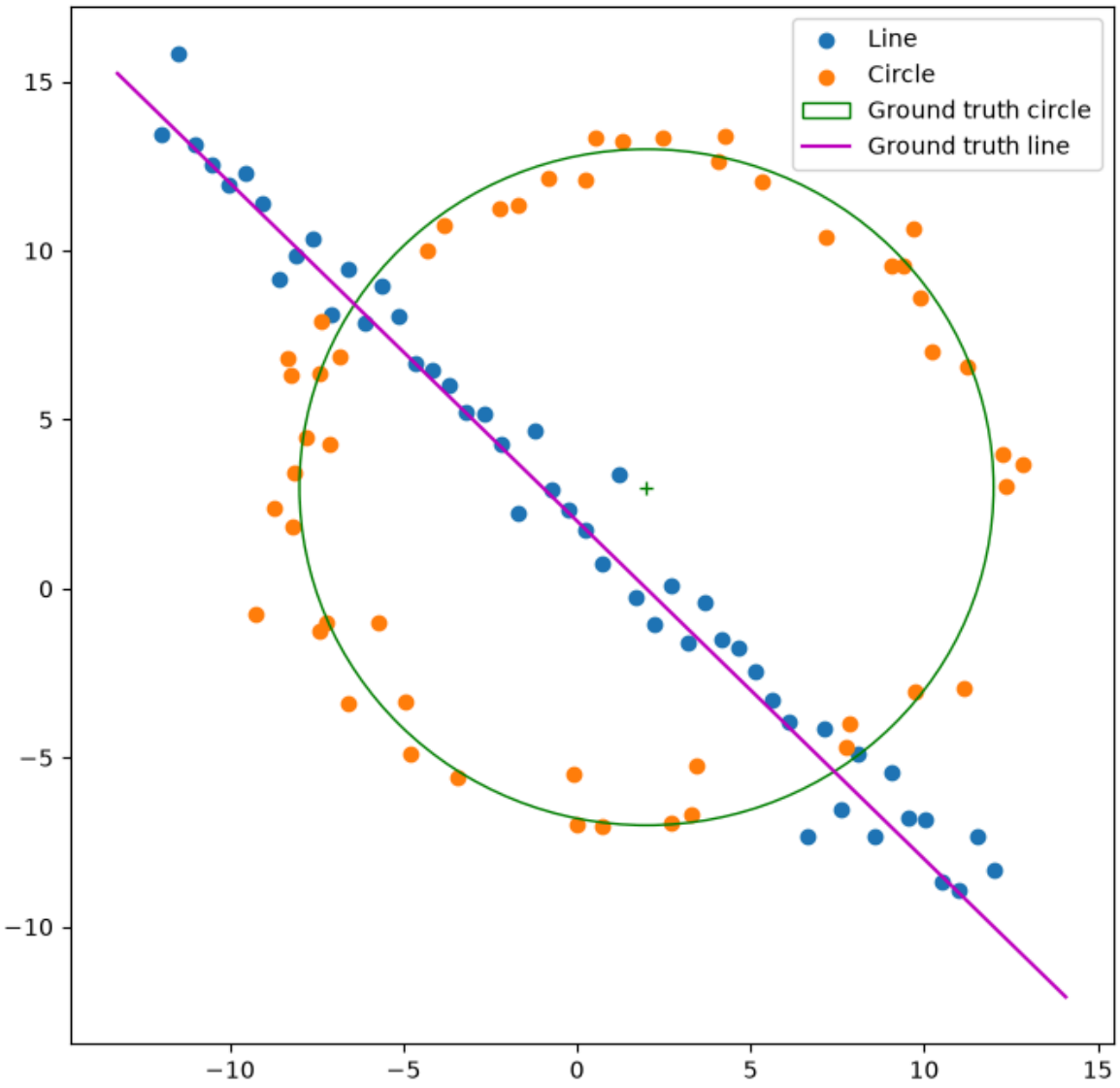
# Plot ground truth circle
circle_gt = plt.Circle(
    (x0_gt, y0_gt),
    r,
    color='g',
    fill=False,
    label='Ground truth circle'
)

ax.add_patch(circle_gt)
ax.plot(x0_gt, y0_gt, '+', color='g')

# Plot ground truth line
x_min, x_max = ax.get_xlim()
x_ = np.array([x_min, x_max])
y_ = m * x_ + b

plt.plot(x_, y_, color='m', label='Ground truth line')

plt.legend()
plt.show()
```



Ransac line fitting

In [10]:

```
def line_from_two(p1, p2):
    direction = p2 - p1
    length = np.linalg.norm(direction)
    if length < 1e-10:
        return None
    a, b = -direction[1] / length, direction[0] / length
    d = a*p1[0] + b*p1[1]
    return np.array([a, b, d])

def line_errors(points, model):
    a, b, d = model
    return np.abs(points[:, 0]*a + points[:, 1]*b - d)

def tls_line(points):
    mean = points.mean(axis=0)
    _, _, vt = np.linalg.svd(points - mean)
    a, b = vt[-1]
    return np.array([a, b, np.dot([a, b], mean)])

rng = np.random.default_rng(5)
line_threshold = 1.2
min_line_inliers = 35
```

```
iterations = 2000
best_line_sample = None
best_line_model = None
best_line_mask = np.zeros(len(X), dtype=bool)

for _ in range(iterations):
    ids = rng.choice(len(X), 2, replace=False)
    model = line_from_two(X[ids[0]], X[ids[1]])
    if model is None:
        continue
    mask = line_errors(X, model) < line_threshold
    if mask.sum() > best_line_mask.sum():
        best_line_mask = mask
        best_line_model = model
        best_line_sample = X[ids].copy()

if best_line_mask.sum() < min_line_inliers:
    raise RuntimeError('No line consensus found. Adjust the threshold.')

line_fit = tls_line(X[best_line_mask])
line_inliers = line_errors(X, line_fit) < line_threshold
print('Line consensus:', best_line_mask.sum())
print('Refitted line (a, b, d):', np.round(line_fit, 3))
print('Unit normal length:', np.linalg.norm(line_fit[:2]))
```

```
Line consensus: 51
Refitted line (a, b, d): [-0.69  -0.724 -1.627]
Unit normal length: 1.0000000000000002
```

In [11]:

```
# Combine all points
X = np.vstack((X_circ, X_line))

# Plot points
fig, ax = plt.subplots(1, 1, figsize=(8, 8))

ax.scatter(X_line[:, 0], X_line[:, 1], label='Line')
ax.scatter(X_circ[:, 0], X_circ[:, 1], label='Circle')

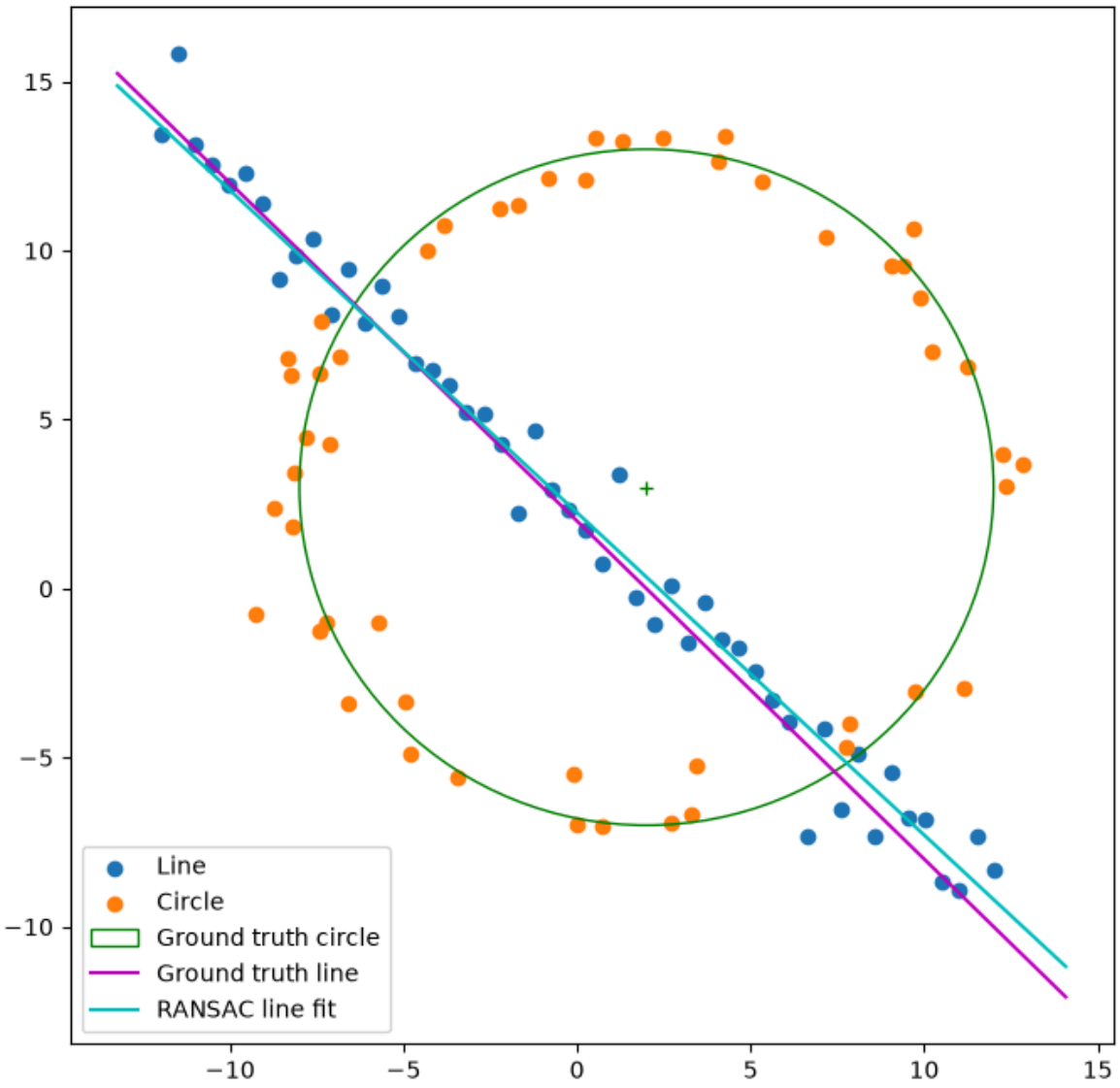
# Plot ground truth circle
circle_gt = plt.Circle(
    (x0_gt, y0_gt),
    r,
    color='g',
    fill=False,
    label='Ground truth circle'
)

ax.add_patch(circle_gt)
ax.plot(x0_gt, y0_gt, '+', color='g')

# Plot ground truth line
x_min, x_max = ax.get_xlim()
x_ = np.array([x_min, x_max])
y_ = m * x_ + b

plt.plot(x_, y_, color='m', label='Ground truth line')
plt.plot(x_, (line_fit[2] - line_fit[0]*x_) / line_fit[1], color='c', lab

plt.legend()
plt.show()
```



Ransac circle fitting

In [12]:

```
def circle_from_three(points):
    p1, p2, p3 = points
    A = 2 * np.array([p2-p1, p3-p1])
    B = np.array([np.dot(p2, p2)-np.dot(p1, p1),
                  np.dot(p3, p3)-np.dot(p1, p1)])
    if abs(np.linalg.det(A)) < 1e-6:
        return None
    center = np.linalg.solve(A, B)
    radius = np.linalg.norm(p1-center)
    return np.array([center[0], center[1], radius])

def circle_errors(points, model):
    xc, yc, radius = model
    return np.abs(np.linalg.norm(points - np.array([xc, yc]), axis=1) - r

remaining = X[~best_line_mask]
circle_threshold = 1.0
min_circle_inliers = 40
best_circle_mask = np.zeros(len(remaining), dtype=bool)
best_circle_model = None
best_circle_sample = None
```

```
for _ in range(iterations):
    ids = rng.choice(len(remaining), 3, replace=False)
    sample = remaining[ids]
    model = circle_from_three(sample)
    if model is None or model[2] > 30:
        continue
    mask = circle_errors(remaining, model) < circle_threshold
    if mask.sum() > best_circle_mask.sum():
        best_circle_mask = mask
        best_circle_model = model
        best_circle_sample = sample.copy()

if best_circle_mask.sum() < min_circle_inliers:
    raise RuntimeError('No circle consensus found. Adjust the threshold.'

def objective(params, pts):
    return np.sum(circle_errors(pts, params)**2)

opt = minimize(objective, best_circle_model,
               args=(remaining[best_circle_mask],),
               method='Nelder-Mead')
circle_fit = opt.x
circle_inliers = circle_errors(remaining, circle_fit) < circle_threshold
print('Circle consensus:', best_circle_mask.sum())
print('Refitted circle (xc, yc, r):', np.round(circle_fit, 3))
```

```
Circle consensus: 43
Refitted circle (xc, yc, r): [ 2.03   2.743 10.103]
```

In [13]:

```
# Combine all points
X = np.vstack((X_circ, X_line))

# Plot points
fig, ax = plt.subplots(1, 1, figsize=(8, 8))

ax.scatter(X_line[:, 0], X_line[:, 1], label='Line')
ax.scatter(X_circ[:, 0], X_circ[:, 1], label='Circle')

# Plot ground truth circle
circle_gt = plt.Circle(
    (x0_gt, y0_gt),
    r,
    color='g',
    fill=False,
    label='Ground truth circle'
)

ax.add_patch(circle_gt)
ax.plot(x0_gt, y0_gt, '+', color='g')

# Plot ground truth line
x_min, x_max = ax.get_xlim()
x_ = np.array([x_min, x_max])
y_ = m * x_ + b

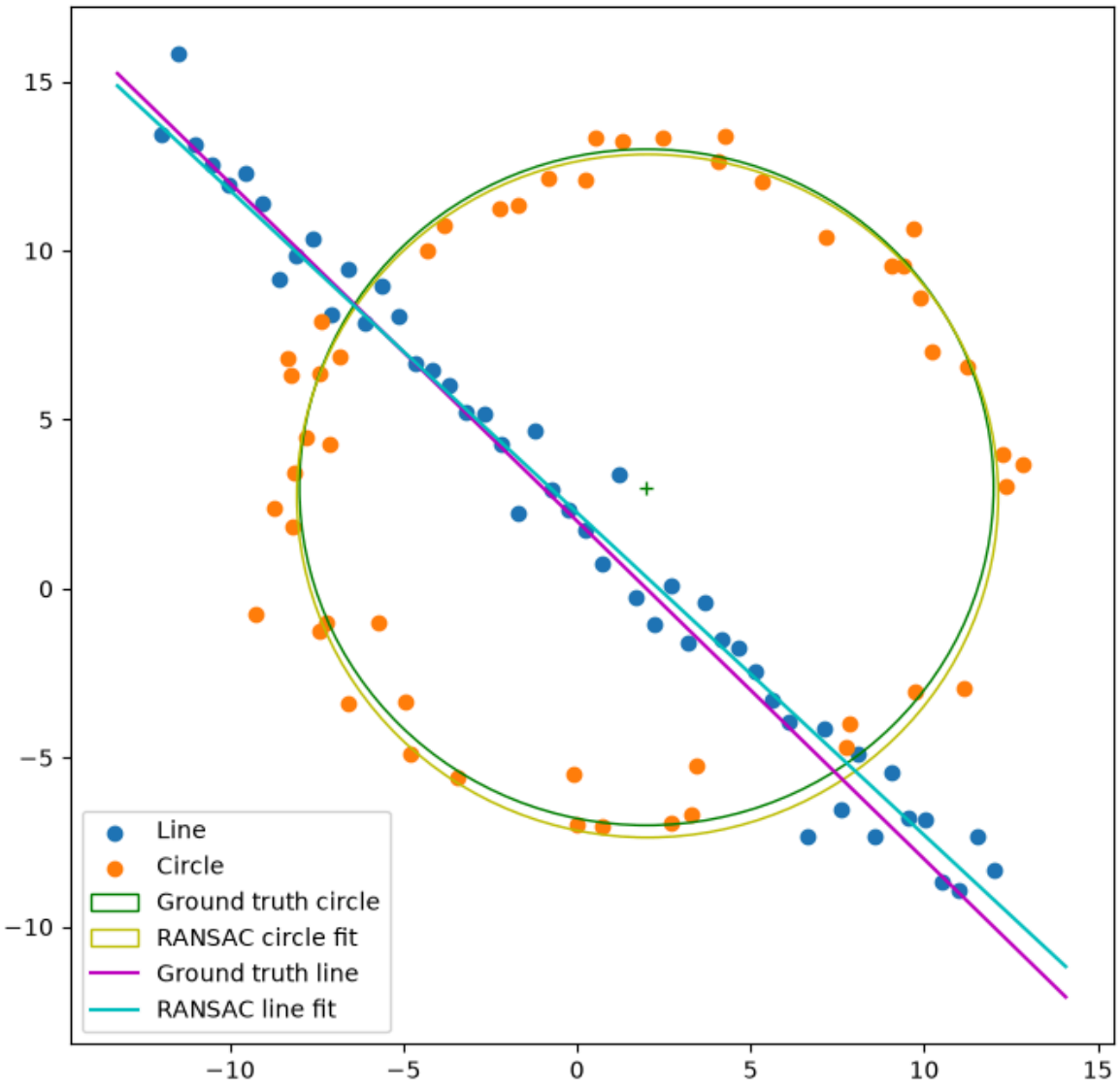
circle = Circle(
    (circle_fit[0], circle_fit[1]),
    circle_fit[2],
    color='y',
    fill=False,
```

```
    label='RANSAC circle fit'
)

plt.gca().add_patch(circle)

plt.plot(x_, y_, color='m', label='Ground truth line')
plt.plot(x_, (line_fit[2] - line_fit[0]*x_) / line_fit[1], color='c', lab

plt.legend()
plt.show()
```



In [14]:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import minimize

np.random.seed(0)


# Circle RANSAC
best_circle_inliers = np.array([], dtype=int)
best_circle_sample = None
best_circle_model = None

for _ in range(2000):
    ids = np.random.choice(len(X), 3, replace=False)
    model = circle_from_three(X[ids])
```

```
    if model is None:
        continue

    inliers = np.where(circle_errors(X, model) < 1.0)[0]

    if len(inliers) > len(best_circle_inliers):
        best_circle_inliers = inliers
        best_circle_model = model
        best_circle_sample = X[ids].copy()

# Refine circle using all inliers
def circle_objective(params):
    return np.sum(circle_errors(X[best_circle_inliers], params)**2)

circle_result = minimize(circle_objective, best_circle_model)
xc, yc, radius = circle_result.x

# Remove circle inliers
remaining_mask = np.ones(len(X), dtype=bool)
remaining_mask[best_circle_inliers] = False
X_remaining = X[remaining_mask]

# Line RANSAC
best_line_inliers = np.array([], dtype=int)
best_line_model = None
best_line_sample = None

for _ in range(2000):
    ids = np.random.choice(len(X_remaining), 2, replace=False)
    model = line_from_two(*X_remaining[ids])

    if model is None:
        continue

    inliers = np.where(line_errors(X_remaining, model) < 1.2)[0]

    if len(inliers) > len(best_line_inliers):
        best_line_inliers = inliers
        best_line_model = model
        best_line_sample = X_remaining[ids].copy()

# Refine line using SVD (total least squares)
line_points = X_remaining[best_line_inliers]
mean = line_points.mean(axis=0)
_, _, Vt = np.linalg.svd(line_points - mean)

a, b = Vt[-1]
d = a * mean[0] + b * mean[1]

# Normalize direction of line normal
norm = np.hypot(a, b)
a, b, d = a / norm, b / norm, d / norm


# Plot
fig, ax = plt.subplots(figsize=(9, 9))

ax.scatter(X[:, 0], X[:, 1], s=15, color='gray', label='All points')
ax.scatter(X[best_circle_inliers, 0], X[best_circle_inliers, 1],
```

```
            s=25, color='orange', label='Circle inliers')
ax.scatter(line_points[:, 0], line_points[:, 1],
            s=25, color='blue', label='Line inliers')

ax.scatter(best_circle_sample[:, 0], best_circle_sample[:, 1],
            s=110, marker='x', color='red', label='Circle sample (3 points
ax.scatter(best_line_sample[:, 0], best_line_sample[:, 1],
            s=110, marker='x', color='purple', label='Line sample (2 point

ax.add_patch(plt.Circle((xc, yc), radius,
                        fill=False, color='red', linewidth=2,
                        label='Best-fit circle'))

ax.add_patch(plt.Circle(best_circle_model[:2], best_circle_model[2],
                        fill=False, color='red', linestyle='--',
                        label='Sample circle'))

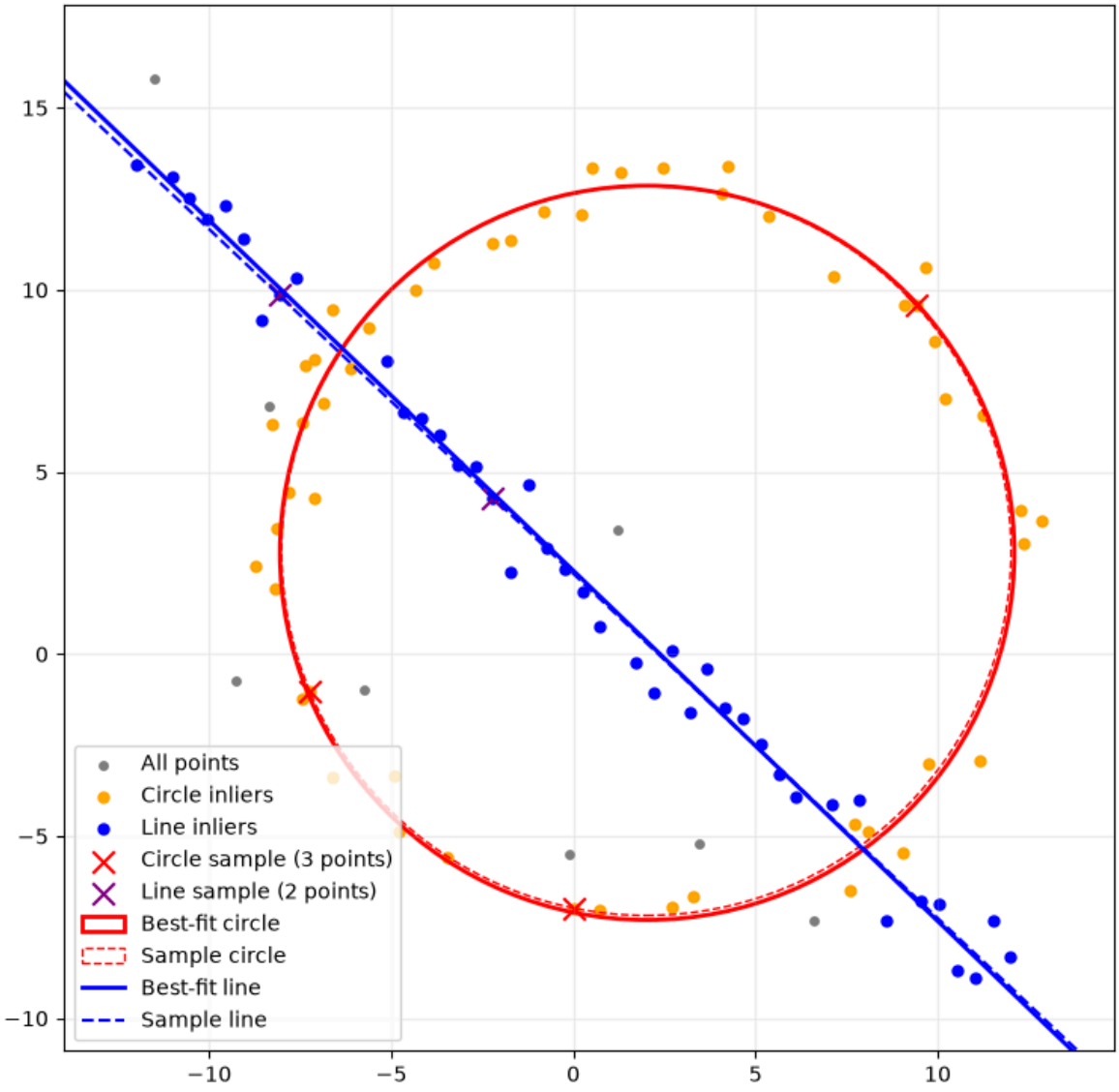
x_plot = np.linspace(X[:, 0].min() - 2, X[:, 0].max() + 2, 200)

if abs(b) > 1e-8:
    y_plot = (d - a * x_plot) / b
    sample_y = (best_line_model[2] - best_line_model[0] * x_plot) / best_

    ax.plot(x_plot, y_plot, 'b-', linewidth=2, label='Best-fit line')
    ax.plot(x_plot, sample_y, 'b--', label='Sample line')

ax.set_aspect('equal')
ax.set_xlim(X[:, 0].min() - 2, X[:, 0].max() + 2)
ax.set_ylim(X[:, 1].min() - 2, X[:, 1].max() + 2)
ax.legend()
ax.grid(alpha=0.2)
plt.show()

print(f"Circle: center=({xc:.3f}, {yc:.3f}), radius={radius:.3f}")
print(f"Circle inliers: {len(best_circle_inliers)}")
print(f"Line: a={a:.3f}, b={b:.3f}, d={d:.3f}")
print(f"Line inliers: {len(best_line_inliers)}")
```




```
Circle: center=(2.025, 2.782), radius=10.081
Circle inliers: 51
Line: a=0.693, b=0.721, d=1.651
Line inliers: 41
```

With the current data, fitting the circle then fitting the line seems to perform better. This can be due the circle data affecting the line data. So By first fitting the circle and removing those data points and fitting the line. Accuaracy of both circle and line goes up.

Question 3

In [15]:
```
# Keep result plots inline; point selection uses a native OpenCV window.
%matplotlib inline
```

In [16]:
```
from pathlib import Path

folder = Path('images')
CLICK_FOUR_POINTS = True  # Set to True to select 4 points with your mous
BLEND_ALPHA = 0.88        # 1.0 = opaque flag, 0.0 = original building

pairs = [
```

```
    ('Modern building + Japan', 'wadham.jpg', 'flag.png',
     [[370, 220], [620, 235], [614, 432], [361, 424]])
]

def read_rgb(path):
    im = cv.imread(str(path))
    if im is None:
        raise FileNotFoundError(f'Image not found: {path}')
    return cv.cvtColor(im, cv.COLOR_BGR2RGB)

def get_four_points(image, preset):
    if not CLICK_FOUR_POINTS:
        return np.float32(preset)

    # OpenCV handles mouse events locally, without VS Code's widget rende
    window = 'Select TL, TR, BR, BL | Enter: confirm | Right-click: undo
    height, width = image.shape[:2]
    scale = min(1.0, 1200 / width, 800 / height)
    display_size = (max(1, round(width * scale)), max(1, round(height * s
    preview = cv.resize(cv.cvtColor(image, cv.COLOR_RGB2BGR), display_siz
    points = []

    def on_click(event, x, y, flags, param):
        if event == cv.EVENT_LBUTTONDOWN and len(points) < 4:
            if 0 <= x < display_size[0] and 0 <= y < display_size[1]:
                points.append((x, y))
        elif event == cv.EVENT_RBUTTONDOWN and points:
            points.pop()

    print('Select TL, TR, BR, BL in the image window, then press Enter. '
          'Right-click to undo; Esc to cancel.')
    cv.namedWindow(window, cv.WINDOW_AUTOSIZE)
    cv.setMouseCallback(window, on_click)
    try:
        while True:
            marked = preview.copy()
            for i, (x, y) in enumerate(points):
                cv.circle(marked, (x, y), 5, (0, 0, 255), -1)
                cv.putText(marked, str(i + 1), (x + 8, y - 8),
                           cv.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)
            cv.imshow(window, marked)
            key = cv.waitKey(20) & 0xFF
            if key == 27 or cv.getWindowProperty(window, cv.WND_PROP_VISI
                raise RuntimeError('Point selection cancelled. Rerun this
            if key in (10, 13) and len(points) == 4:
                break
    finally:
        cv.destroyWindow(window)

    # Map preview coordinates back to the original image for the homograp
    dst = np.asarray(points, dtype=np.float32)
    dst *= np.float32([width / display_size[0], height / display_size[1]]
    print('Selected points:')
    print(dst)
    return dst
```

In [17]:

```
def overlay_flag(building, flag, corners, alpha=0.88):
    h, w = flag.shape[:2]
    src = np.float32([[0, 0], [w-1, 0], [w-1, h-1], [0, h-1]])
```

```
    H = cv.getPerspectiveTransform(src, corners)
    out_size = (building.shape[1], building.shape[0])
    warped = cv.warpPerspective(flag, H, out_size)

    mask = np.full((h, w), 255, dtype=np.uint8)
    warped_mask = cv.warpPerspective(mask, H, out_size) / 255.0
    a = alpha * warped_mask[..., None]
    output = ((1-a)*building + a*warped).astype(np.uint8)
    return output, H
```

In [18]:

```
results = []

for title, building_name, flag_name, preset in pairs:
    building = read_rgb(folder / building_name)
    flag = read_rgb(folder / flag_name)
    corners = get_four_points(building, preset)
    output, H = overlay_flag(building, flag, corners, 0.5)
    results.append((title, building, flag, corners, output, H))

fig, axes = plt.subplots(len(results), 3, figsize=(17, 4.5 * len(results)
for i, (title, building, flag, corners, output, H) in enumerate(results):
    marked = building.copy()
    cv.polylines(marked, [corners.astype(np.int32)], True, (255, 230, 0),
    for j, (x, y) in enumerate(corners.astype(int)):
        cv.circle(marked, (x, y), 7, (255, 60, 20), -1)
        cv.putText(marked, str(j+1), (x+9, y-9), cv.FONT_HERSHEY_SIMPLEX,
                   0.8, (255, 255, 255), 2)
    axes[i, 0].imshow(marked)
    axes[i, 0].set_title(f'{title}: selected 4 points')
    axes[i, 1].imshow(flag)
    axes[i, 1].set_title('Flag source image')
    axes[i, 2].imshow(output)
    axes[i, 2].set_title('Warped and blended result')
    for ax in axes[i]:
        ax.axis('off')
plt.tight_layout()
plt.show()
```

```
Select TL, TR, BR, BL in the image window, then press Enter. Right-click t
o undo; Esc to cancel.
```

```
Warning: Ignoring XDG_SESSION_TYPE=wayland on Gnome. Use QT_QPA_PLATFORM=w
ayland to run on Wayland anyway.
QFontDatabase: Cannot find font directory /home/nilesh/miniconda3/envs/tf2
20/lib/python3.12/site-packages/cv2/qt/fonts.
Note that Qt no longer ships fonts. Deploy some (from https://dejavu-font
s.github.io/ for example) or switch to fontconfig.
QFontDatabase: Cannot find font directory /home/nilesh/miniconda3/envs/tf2
20/lib/python3.12/site-packages/cv2/qt/fonts.
Note that Qt no longer ships fonts. Deploy some (from https://dejavu-font
s.github.io/ for example) or switch to fontconfig.
QFontDatabase: Cannot find font directory /home/nilesh/miniconda3/envs/tf2
20/lib/python3.12/site-packages/cv2/qt/fonts.
Note that Qt no longer ships fonts. Deploy some (from https://dejavu-font
s.github.io/ for example) or switch to fontconfig.
QFontDatabase: Cannot find font directory /home/nilesh/miniconda3/envs/tf2
20/lib/python3.12/site-packages/cv2/qt/fonts.
Note that Qt no longer ships fonts. Deploy some (from https://dejavu-font
s.github.io/ for example) or switch to fontconfig.
QFontDatabase: Cannot find font directory /home/nilesh/miniconda3/envs/tf2
20/lib/python3.12/site-packages/cv2/qt/fonts.
Note that Qt no longer ships fonts. Deploy some (from https://dejavu-font
s.github.io/ for example) or switch to fontconfig.
Selected points:
[[146. 205.]
 [519. 289.]
 [518. 516.]
 [132. 520.]]
```



Question 4

In [19]:
```python
# Use the images/ folder if the template is there; otherwise the current
IMAGE_DIR = Path("images/pcbs")
OUTPUT_DIR = Path("pcb_outputs")
OUTPUT_DIR.mkdir(exist_ok=True)

reference_path = IMAGE_DIR / "AECC1.jpg"
template = cv.imread(str(reference_path))
if template is None:
    raise FileNotFoundError(f"Template not found: {reference_path.resolve

image_files = sorted(
    p for p in IMAGE_DIR.iterdir()
    if p.is_file() and p.suffix.lower() in (".jpg")
    and p.name.lower() != "aecc1.jpg"
)

print("Template:", reference_path)
print("PCB images:", [p.name for p in image_files])
```

```
Template: images/pcbs/AECC1.jpg
PCB images: ['AAAA1.jpg', 'ADAD1.jpg', 'AECC2.jpg', 'AECC3.jpg', 'AECC4.jp
g']
```

In [20]:
```python
def align_and_compare(template, pcb):
    gray_ref = cv.cvtColor(template, cv.COLOR_BGR2GRAY)
    gray_pcb = cv.cvtColor(pcb, cv.COLOR_BGR2GRAY)

    sift = cv.SIFT_create()
    kp_ref, des_ref = sift.detectAndCompute(gray_ref, None)
    kp_pcb, des_pcb = sift.detectAndCompute(gray_pcb, None)

    if des_ref is None or des_pcb is None:
        raise ValueError("Could not detect SIFT descriptors in both image

    matcher = cv.BFMatcher(cv.NORM_L2)
    pairs = matcher.knnMatch(des_ref, des_pcb, k=2)
    good = [m for pair in pairs if len(pair) == 2
            for m, n in [pair] if m.distance < 0.75 * n.distance]

    if len(good) < 4:
        raise ValueError(f"Not enough good matches to estimate homography

    src = np.float32([kp_pcb[m.trainIdx].pt for m in good]).reshape(-1, 1
    dst = np.float32([kp_ref[m.queryIdx].pt for m in good]).reshape(-1, 1
    H, mask = cv.findHomography(src, dst, cv.RANSAC, 50.0)

    if H is None or mask is None:
        raise ValueError("RANSAC could not estimate a homography.")

    inlier_matches = [m for m, accepted in zip(good, mask.ravel()) if acc
    match_image = cv.drawMatches(template, kp_ref, pcb, kp_pcb, inlier_ma
                                 None, flags=cv.DrawMatchesFlags_NOT_DRAW

    height, width = gray_ref.shape
    aligned = cv.warpPerspective(pcb, H, (width, height))

    # Valid image area after warping: prevents artificial black-border di
    valid = cv.warpPerspective(np.full(gray_pcb.shape, 255, np.uint8), H,
                               (width, height), flags=cv.INTER_NEAREST) >
    aligned_gray = cv.cvtColor(aligned, cv.COLOR_BGR2GRAY)
    difference = cv.absdiff(gray_ref, aligned_gray)
    difference[~valid] = 0

    return match_image, aligned, difference, len(good), len(inlier_matche
```

In [21]:
```python
results = []

for path in image_files:
    pcb = cv.imread(str(path))
    if pcb is None:
        print(f"Skipping unreadable image: {path.name}")
        continue

    try:
        matches, aligned, difference, n_good, n_inliers, valid = align_an
    except ValueError as error:
        print(f"Skipping {path.name}: {error}")
        continue
```

```
    name = path.stem
    cv.imwrite(str(OUTPUT_DIR / f"{name}_matches.jpg"), matches)
    cv.imwrite(str(OUTPUT_DIR / f"{name}_aligned.jpg"), aligned)
    cv.imwrite(str(OUTPUT_DIR / f"{name}_difference.png"), difference)

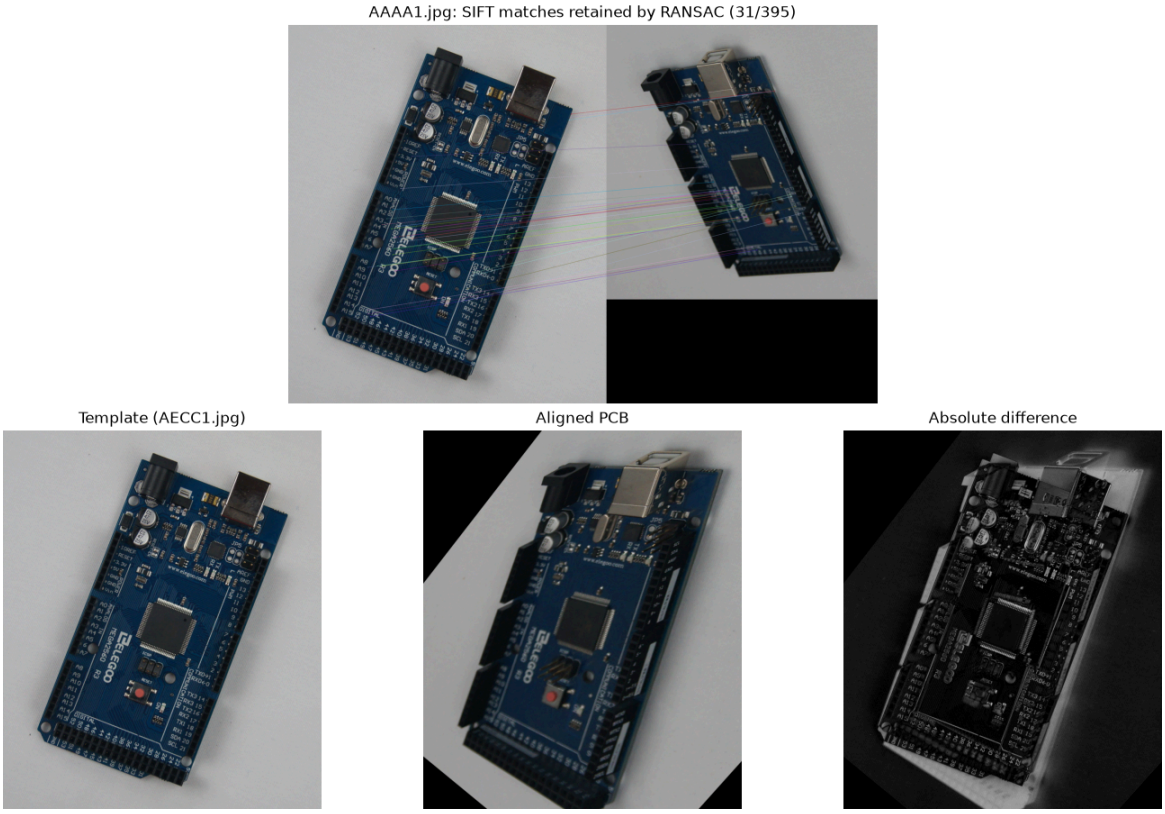
    mean_difference = float(difference[valid].mean()) if valid.any() else
    results.append((path.name, n_good, n_inliers, mean_difference))

    fig = plt.figure(figsize=(14, 9), constrained_layout=True)
    gs = fig.add_gridspec(2, 3)
    ax = fig.add_subplot(gs[0, :])
    ax.imshow(cv.cvtColor(matches, cv.COLOR_BGR2RGB))
    ax.set_title(f"{path.name}: SIFT matches retained by RANSAC ({n_inlie
    ax.axis("off")

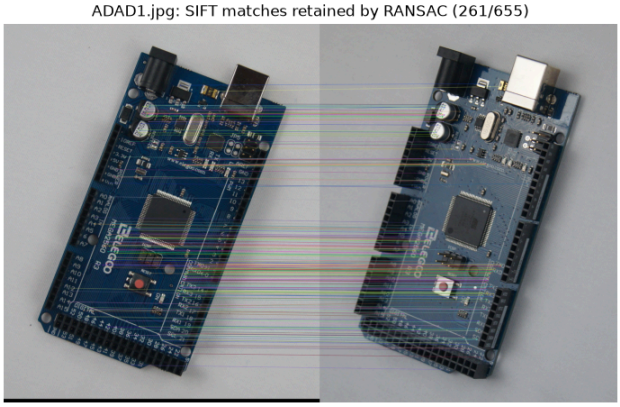
    for position, (img, title, args) in enumerate([
        (template, "Template (AECC1.jpg)", {}),
        (aligned, "Aligned PCB", {}),
        (difference, "Absolute difference", {"cmap": "gray", "vmin": 0, "
    ]):
        ax = fig.add_subplot(gs[1, position])
        if img.ndim == 3:
            ax.imshow(cv.cvtColor(img, cv.COLOR_BGR2RGB))
        else:
            ax.imshow(img, **args)
        ax.set_title(title)
        ax.axis("off")

    plt.show()

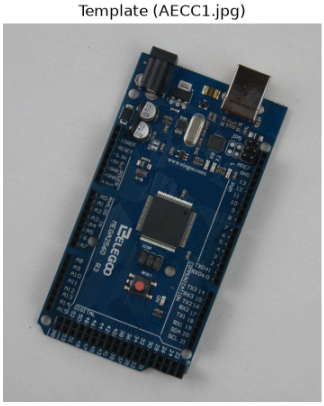
print("\nSummary (mean difference computed only where the warped image is
print(f"{'Image':28} {'Good matches':>12} {'RANSAC inliers':>15} {'Mean d
for filename, n_good, n_inliers, mae in results:
    print(f"{filename[:28]:28} {n_good:12d} {n_inliers:15d} {mae:12.2f}")
print("\nSaved images to:", OUTPUT_DIR.resolve())
```



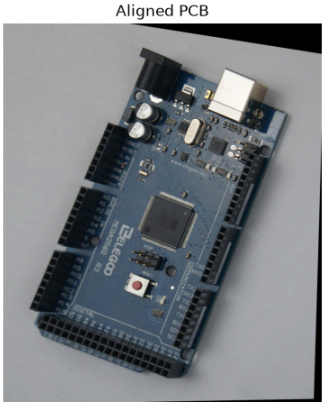
ADAD1.jpg: SIFT matches retained by RANSAC (261/655)



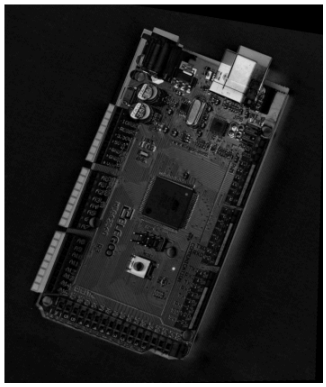
Template (AECC1.jpg)



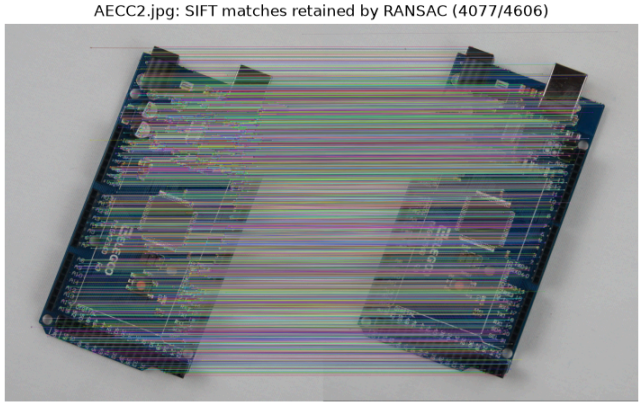
Aligned PCB



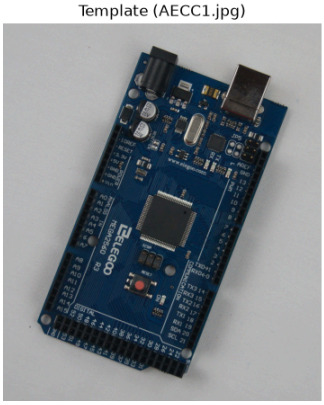
Absolute difference



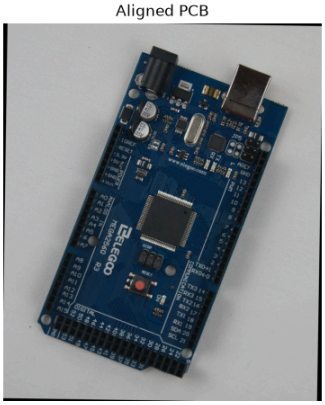
AECC2.jpg: SIFT matches retained by RANSAC (4077/4606)



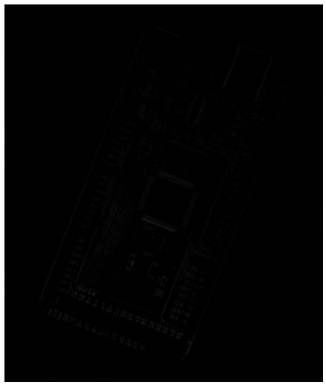
Template (AECC1.jpg)



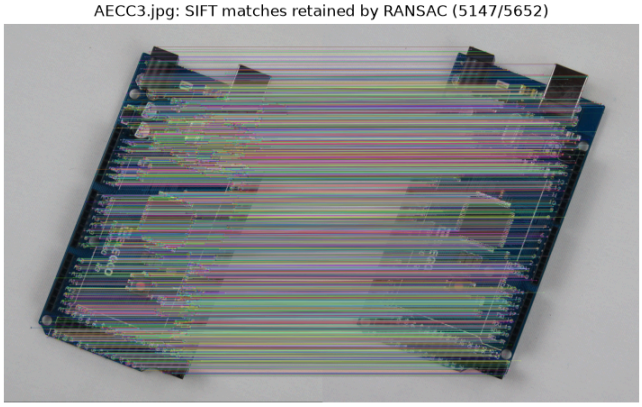
Aligned PCB



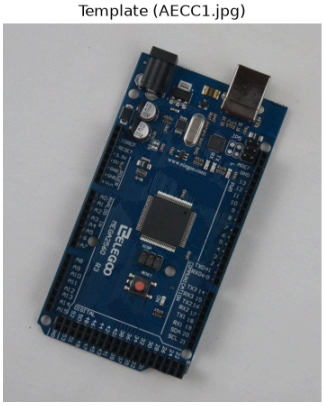
Absolute difference



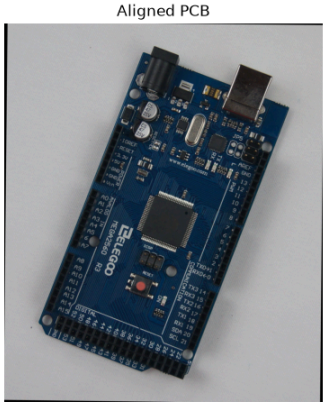
AECC3.jpg: SIFT matches retained by RANSAC (5147/5652)



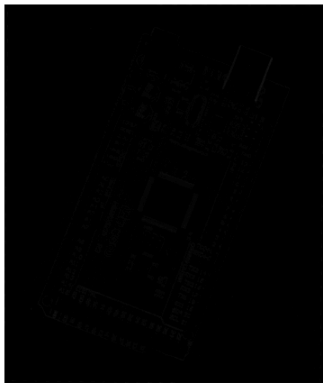
Template (AECC1.jpg)



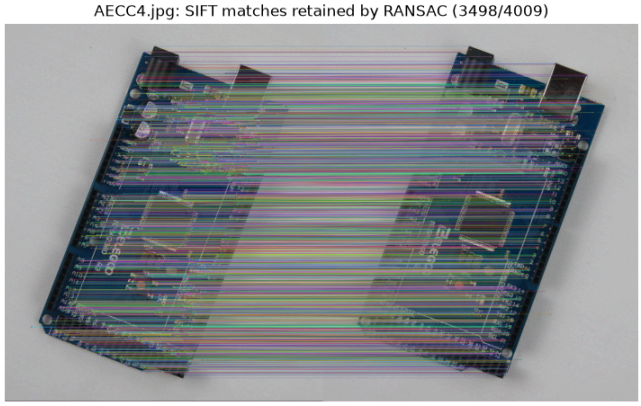
Aligned PCB



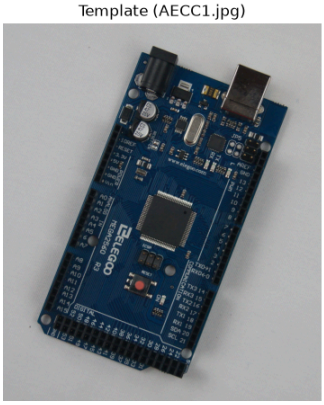
Absolute difference



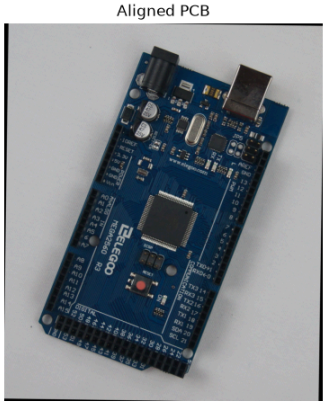
AECC4.jpg: SIFT matches retained by RANSAC (3498/4009)



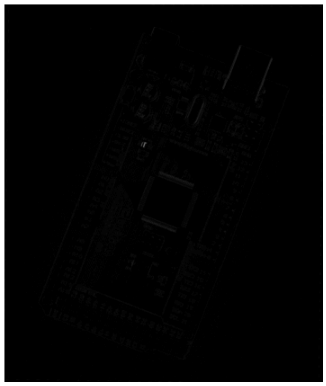
Template (AECC1.jpg)



Aligned PCB



Absolute difference



Summary (mean difference computed only where the warped image is valid)

Image	Good matches	RANSAC inliers	Mean diff.
AAAA1.jpg	395	31	33.99
ADAD1.jpg	655	261	31.86
AECC2.jpg	4606	4077	3.14
AECC3.jpg	5652	5147	2.94
AECC4.jpg	4009	3498	3.63

Saved images to: /home/nilesh/UOM/CV/Assignments/A02/pcb_outputs

USB port, capacitors, resistors and buttons show a high difference compared to the reference image. The main reason for this is the the parallax introduce by the camera movement. It is evident by the images `AAAA1.jpg` and `ADAD1.jpg`, while other have only rotational and a small parallax movement, these 2 images have heavy parallax. This introduces the hights of the component which is not present in the reference image leading to higher differences.